

Hands-On Exercise 1: First Web API using .NET Core

Scenario

The objective of this exercise is to create a simple ASP.NET Core Web API application using the Web API template. The application demonstrates the basic structure of a RESTful Web API and implements CRUD operations using HTTP action verbs.

Objective

- Create a .NET Core Web API project.
- Understand the structure of a Web API.
- Create a controller with Read and Write action methods.
- Implement HTTP action verbs (GET, POST, PUT, DELETE).
- Execute the Web API using Swagger UI.

Project Name

FirstWebAPI

Software Requirements

- Visual Studio 2022
- .NET 8.0 SDK
- ASP.NET Core Web API
- Swagger UI

Implementation

Step 1: Create a New ASP.NET Core Web API Project

1. Open **Visual Studio 2022**.
2. Click **Create a new project**.
3. Search for **ASP.NET Core Web API**.
4. Select the template and click **Next**.
5. Enter the project name as **FirstWebAPI**.
6. Click **Next**.
7. Select **.NET 8.0** as the target framework.
8. Keep **Use Controllers** enabled.
9. Enable **OpenAPI (Swagger)**.

10. Click **Create**.

Step 2: Project Structure

After creating the project, the following folders and files were generated automatically.

```
FirstWebAPI
|
|— Controllers
|   |— WeatherForecastController.cs
|   └─ ValuesController.cs
|
|— Properties
|   └─ launchSettings.json
|
|— appsettings.json
|— appsettings.Development.json
|— Program.cs
└─ FirstWebAPI.csproj
```

Step 3: Create ValuesController

Inside the **Controllers** folder:

- Right-click **Controllers**
- Select **Add → Controller**
- Choose **API Controller - Empty**
- Name the controller **ValuesController.cs**

Step 4: Implement Action Methods

The ValuesController was implemented with the following HTTP action methods.

using Microsoft.AspNetCore.Mvc;

```
namespace FirstWebAPI.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class ValuesController : ControllerBase
    {
        [HttpGet]
        public IActionResult Get()
        {
            return Ok(new string[] { "Apple", "Banana", "Orange" });
        }
    }
}
```

```

    }

    [HttpGet("{id}")]
    public IActionResult Get(int id)
    {
        return Ok($"Value : {id}");
    }

    [HttpPost]
    public IActionResult Post([FromBody] string value)
    {
        return Ok("Record Inserted Successfully");
    }

    [HttpPut("{id}")]
    public IActionResult Put(int id, [FromBody] string value)
    {
        return Ok("Record Updated Successfully");
    }

    [HttpDelete("{id}")]
    public IActionResult Delete(int id)
    {
        return Ok("Record Deleted Successfully");
    }
}
}

```

Step 5: Build the Project

The project was built successfully using:

Build → Rebuild Solution

The build completed successfully without any errors.

Step 6: Run the Application

The application was executed using:

Ctrl + F5

After running the project, **Swagger UI** opened automatically in the browser.

Example URL:

<https://localhost:7086/swagger/index.html>

Step 7: Verify API Endpoints

Swagger UI displayed the following API endpoints.

```
GET    /api/Values
POST   /api/Values
GET    /api/Values/{id}
PUT    /api/Values/{id}
DELETE /api/Values/{id}
```

The default **WeatherForecast** endpoint was also available.

Step 8: Test the GET Method

The **GET /api/Values** endpoint was tested using Swagger.

Steps performed:

1. Expand **GET /api/Values**.
2. Click **Try it Out**.
3. Click **Execute**.

Expected Response:

```
[
  "Apple",
  "Banana",
  "Orange"
]
```

This confirms that the API is working correctly.

HTTP Action Verbs Used

Action Verb Purpose

GET	Retrieves data from the server.
POST	Inserts new data.
PUT	Updates existing data.
DELETE	Deletes existing data.

HTTP Status Code Used

The API methods return:

```
return Ok(...);
```

which corresponds to the HTTP status code:

200 OK

Result

A simple **ASP.NET Core Web API** application was successfully created using the Web API template. A ValuesController was implemented with **GET**, **POST**, **PUT**, and **DELETE** action methods. The application executed successfully, Swagger UI displayed all API endpoints, and the GET endpoint returned the expected response, demonstrating the successful implementation of a RESTful Web API.



